Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:19/03/2026**	**Enrollment No:92301733054**

Aim: To obtain the classification of various classes using k-Nearest Neighbor approach

IDE: Google Colab

Theory:

This algorithm is used to solve the classification model problems. K-nearest neighbor or K-NN algorithm basically creates an imaginary boundary to classify the data. When new data points come in, the algorithm will try to predict that to the nearest of the boundary line. Therefore, larger k value means smother curves of separation resulting in less complex models. Whereas, smaller k value tends to over fit the data and resulting in complex models. It's very important to have the right k-value when analyzing the dataset to avoid over fitting and under fitting of the dataset.

The model representation for KNN is the entire training dataset. It is as simple as that. KNN has no model other than storing the entire dataset, so there is no learning required. Efficient implementations can store the data using complex data structures like k-d trees to make look-up and matching of new patterns during prediction efficient. Because the entire training dataset is stored, you may want to think carefully about the consistency of your training data. It might be a good idea to curate it, update it often as new data becomes available and remove erroneous and outlier data.

K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories. K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm. K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data. It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset. KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Pre-process the data

5. Visualize the Data
6. Separate the feature and prediction value columns
7. Select the number K of the neighbors
8. Calculate the Euclidean distance of K number of neighbors
9. Take the K nearest neighbors as per the calculated Euclidean distance.
10. Among these k neighbors, count the number of the data points in each category.
11. Assign the new data points to that category for which the number of the neighbor is maximum.
12. Our model is ready.

Pre Lab Exercise:

a. What is K in KNN-approach?

- K represents the number of nearest neighbors considered for classification
- It determines how many nearby data points influence the prediction
- Example: K = 5 → consider 5 nearest neighbors

b. Which are the types of distance metrices?

- Euclidean Distance
- Manhattan Distance
- Minkowski Distance
- Hamming Distance

c. What can be the criteria of selection of the value of K?

- Small K → may cause overfitting
- Large K → may cause underfitting

d. What are the advantages of KNN algorithm?

- Simple and easy to understand
- No training phase
- Works well for small datasets
- Can be used for both classification and regression

Program (Code):

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:19/03/2026**	**Enrollment No:92301733054**

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

digits = load_digits()
X = digits.data
y = digits.target
images = digits.images

plt.figure(figsize=(8, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(f"{y[i]}")
    plt.axis("off")
plt.suptitle("Sample Digits")
plt.tight_layout()
plt.show()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
knn_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())
])

knn_param_grid = {
    "knn__n_neighbors": [3, 5, 7, 9, 13, 15, 20]}
rf_pipeline = Pipeline([
```

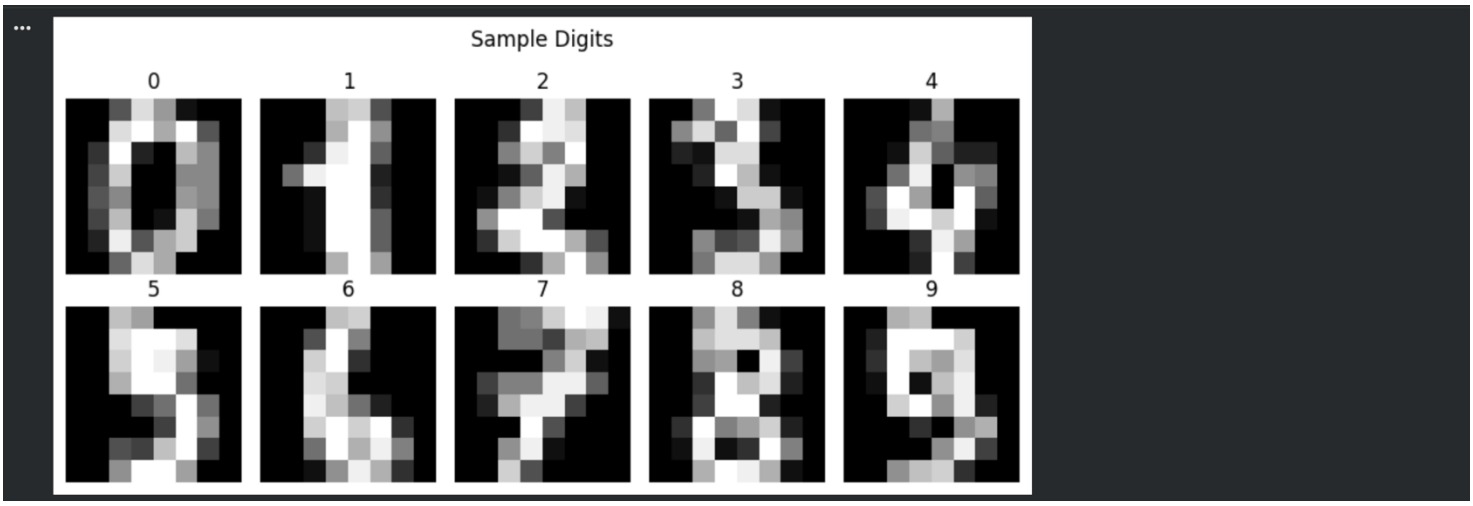
```
    ("rf", RandomForestClassifier(random_state=42))
])

rf_param_grid = {
    "rf__n_estimators": [25, 50, 100, 200, 500],
    "rf__max_depth": [None, 10, 20]}

mlp_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPClassifier(
        max_iter=1000,
        random_state=42,
        early_stopping=True
    ))
])

mlp_param_grid = {
    "mlp__hidden_layer_sizes": [(64,), (128,), (64, 64)],
    "mlp__activation": ["relu", "tanh"],
    #"mlp__alpha": [0.0001, 0.001],
    "mlp__learning_rate_init": [0.001, 0.01, 1.0, 5.0, 10.0]
}

#Training the model
def train_with_timing(pipeline, param_grid):
    start = time.time()
    grid = GridSearchCV(
        pipeline,
        param_grid,
        cv=5,
        scoring="f1_weighted",
        n_jobs=-1
    )
    grid.fit(X_train, y_train)
    train_time = time.time() - start
    return grid, train_time

knn_grid, knn_train_time = train_with_timing(knn_pipeline, knn_param_grid)
rf_grid, rf_train_time = train_with_timing(rf_pipeline, rf_param_grid)
mlp_grid, mlp_train_time = train_with_timing(mlp_pipeline, mlp_param_grid)
#Evaluation
def evaluate_model(model):
    y_pred = model.predict(X_test)
    return {
```

```
    "Accuracy": accuracy_score(y_test, y_pred),
    "Precision": precision_score(y_test, y_pred, average="weighted"),
    "Recall": recall_score(y_test, y_pred, average="weighted"),
    "F1-score": f1_score(y_test, y_pred, average="weighted")
  }

knn_metrics = evaluate_model(knn_grid.best_estimator_)
rf_metrics = evaluate_model(rf_grid.best_estimator_)
mlp_metrics = evaluate_model(mlp_grid.best_estimator_)
#Result with comparison
results_df = pd.DataFrame(
  [knn_metrics, rf_metrics, mlp_metrics],
  index=["KNN", "Random Forest", "MLP"]
)

results_df["Training Time (s)"] = [
  knn_train_time, rf_train_time, mlp_train_time
]

results_df["Inference Time (s)"] = [
  knn_test_time, rf_test_time, mlp_test_time
]

print(results_df)
results_df.iloc[:, :4].plot(kind="bar", figsize=(10, 6))
plt.title("Model Performance Comparison (Digits)")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.grid(axis="y")
plt.show()

#Inferencing the model
def inference_time(model):
  start = time.time()
  model.predict(X_test)
  return time.time() - start

knn_test_time = inference_time(knn_grid.best_estimator_)
rf_test_time = inference_time(rf_grid.best_estimator_)
mlp_test_time = inference_time(mlp_grid.best_estimator_)
#Evaluation
def evaluate_model(model):
  y_pred = model.predict(X_test)
```

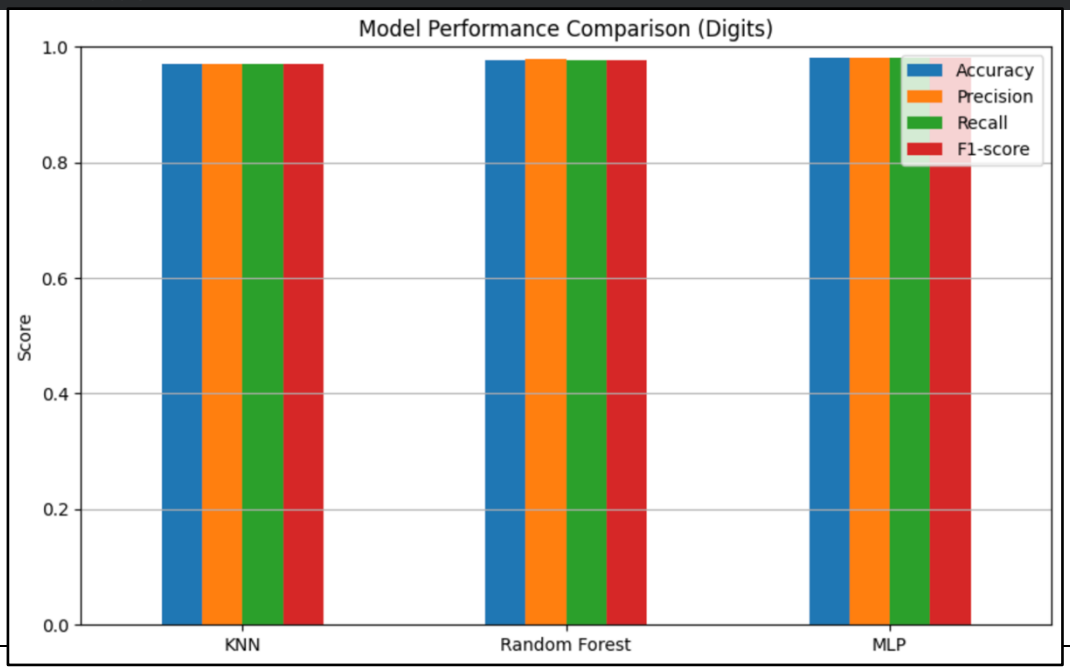
```
    return {
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, average="weighted"),
        "Recall": recall_score(y_test, y_pred, average="weighted"),
        "F1-score": f1_score(y_test, y_pred, average="weighted")
    }

knn_metrics = evaluate_model(knn_grid.best_estimator_)
rf_metrics = evaluate_model(rf_grid.best_estimator_)
mlp_metrics = evaluate_model(mlp_grid.best_estimator_)
#Result with comparison
results_df = pd.DataFrame(
    [knn_metrics, rf_metrics, mlp_metrics],
    index=["KNN", "Random Forest", "MLP"]
)

results_df["Training Time (s)"] = [
    knn_train_time, rf_train_time, mlp_train_time
]

results_df["Inference Time (s)"] = [
    knn_test_time, rf_test_time, mlp_test_time
]

print(results_df)
results_df.iloc[:, :4].plot(kind="bar", figsize=(10, 6))
plt.title("Model Performance Comparison (Digits)")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.grid(axis="y")
plt.show()
models = {
    "KNN": knn_grid.best_estimator_,
    "Random Forest": rf_grid.best_estimator_,
    "MLP": mlp_grid.best_estimator_
}

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (name, model) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, model.predict(X_test))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax)
    ax.set_title(name)
```

```
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")

plt.tight_layout()
plt.show()
```

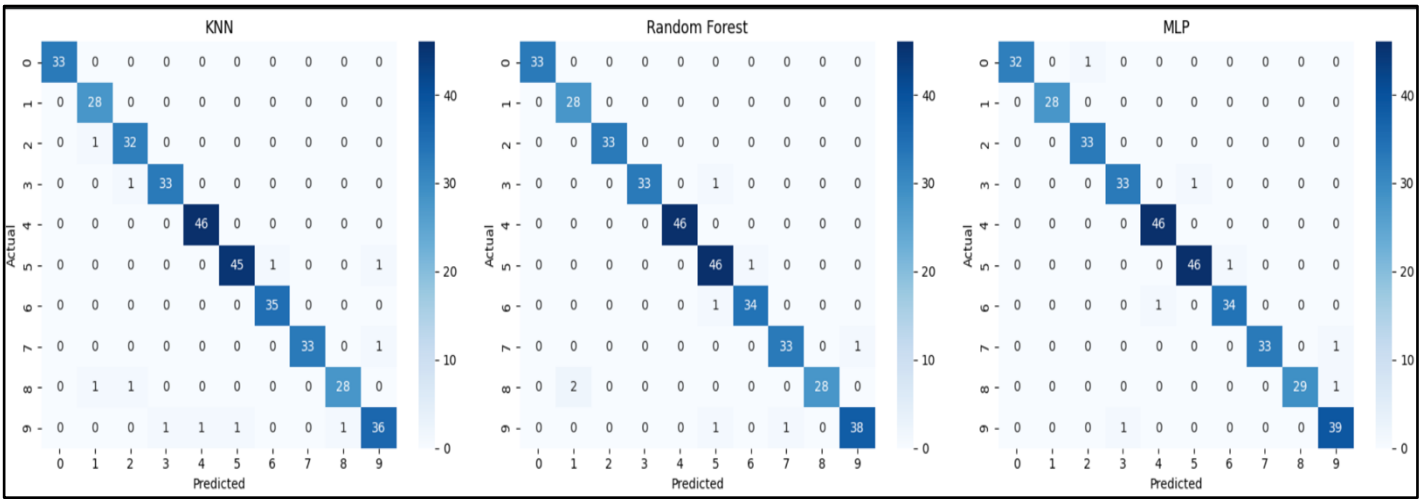
Results:




```
               Accuracy  Precision    Recall  F1-score  Training Time (s)  \
KNN            0.969444   0.969666  0.969444  0.969287           6.105988
Random Forest  0.977778   0.978417  0.977778  0.977820          73.241260
MLP            0.980556   0.980832  0.980556  0.980577          34.318433

               Inference Time (s)
KNN                      0.046493
Random Forest            0.071715
MLP                      0.001977
```



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:19/03/2026**	**Enrollment No:92301733054**



Observation and Result Analysis:

a. Nature of the dataset

- The dataset used is Digits dataset (sklearn)
- It contains image data (8×8 pixels) converted into numerical features
- Total 1797 samples with 64 features each
- Multi-class classification problem (digits 0–9)
- Data is balanced and structured

b. During Training Process

- GridSearchCV is used to find best parameters
- Data is scaled using StandardScaler
- Cross-validation ensures better generalization

c. After the training Process

- KNN gives good accuracy but slower inference
- Random Forest & MLP may give higher accuracy
- Confusion matrix shows classification performance for each digit

d. Observation over the Learning Curve

- Proper K gives optimal performance
- Too small K → high variance
- Too large K → high bias

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:19/03/2026**	**Enrollment No:92301733054**

Post Lab Exercise:

a. Is KNN useful in regression-based problems?

- Yes, KNN can be used for regression problems. Instead of majority voting (as in classification), it predicts the output by taking the average of the K nearest neighbors. This is known as KNN Regression.

b. Why KNN is a non-parametric algorithm?

- KNN is called a non-parametric algorithm because it does not assume any underlying distribution or mathematical model for the data. It simply stores the dataset and makes predictions based on similarity.

c. What are the assumptions of KNN approach?

- The main assumptions of KNN are:
- Similar data points exist close to each other in feature space
- Distance metric correctly measures similarity

d. How can the KNN approach make the prediction of the unseen dataset?

- KNN predicts unseen data using the following steps:
- Calculate the distance between new data point and all training points
- Select the K nearest neighbors
- Count the number of data points in each class
- Assign the class with the maximum votes

e. Why is KNN called a Lazy Learner?

- KNN is called a lazy learner because it does not learn a model during training. It simply stores the entire dataset and performs computation only at the time of prediction.

f. Why should we not use KNN for large dataset?

- KNN is not suitable for large datasets because:
- It requires high memory to store all data
- Prediction is slow